A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

PYTHON MINI PROJECT REPORT

WEBSITE DATA TRACKER

A desktop application for organizing website records

Project Detail	Information
Submitted By	Modh Bhavi
Enrollment No.	12502080603003
Faculty Guide	Mr. Nayan Mali
Technology	Python 3 and Tkinter
Core Concepts	Lists, Dictionaries, File Handling, Tkinter

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the mini project report entitled “Website Data Tracker” has been prepared and submitted by Modh Bhavi (Enrollment No. 12502080603003), Department of Information Technology, A D Patel Institute of Technology.

The project demonstrates Python programming concepts including Lists, Dictionaries, File Handling, and Tkinter for developing a simple graphical user interface. The work has been carried out under the guidance of Mr. Nayan Mali.

Faculty Guide Signature: _____

Name: Mr. Nayan Mali

Department Seal / Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing me the opportunity to complete this mini project.

I am thankful to my faculty guide, Mr. Nayan Mali, for his guidance, encouragement, and valuable suggestions during the development of the Website Data Tracker. I also thank my teachers, classmates, and everyone who directly or indirectly supported me.

Modh Bhavi
Enrollment No.: 12502080603003

ABSTRACT

Website Data Tracker is a Python desktop application for keeping website links and related information organized in one place. Users can enter a website name, URL, category, and notes, then add, view, search, and delete records through a Tkinter interface.

Lists store the collection of website records, Dictionaries represent each record as named fields, and File Handling saves the information to a JSON file so it remains available after the program closes. Tkinter provides the window, input fields, buttons, search box, and table. This educational prototype manages manually entered records; it does not automatically browse websites or monitor visits.

TABLE OF CONTENTS

- 1. Introduction and Problem Statement
- 2. Objectives and Scope
- 3. Requirements
- 4. Python Concepts Used
- 5. System Design and Algorithm
- 6. Source Code
- 7. Testing
- 8. Advantages and Limitations
- 9. Future Enhancements
- 10. Conclusion and References

1. INTRODUCTION AND PROBLEM STATEMENT

People collect useful website links for education, research, communication, shopping, and entertainment. Links scattered across notes, messages, and browser tabs can be difficult to find later. Website Data Tracker offers a small local application to store each website with a name, URL, category, and notes.

The problem addressed is the lack of a simple, structured place to maintain a personal website list. The application provides basic record-management operations using core Python concepts.

2. OBJECTIVES AND SCOPE

- Create a user-friendly graphical interface with Tkinter.
- Add website name, URL, category, and notes.
- Represent records with Dictionaries and manage them in a List.
- Save and load records using JSON file handling.
- Search records and delete a selected record.
- Keep the application simple and suitable for a Python mini project.

Scope: the first version stores data locally on one computer. It does not check whether a website is online, sync records between devices, or provide multiple-user access.

3. HARDWARE AND SOFTWARE REQUIREMENTS

Requirement	Specification
Operating System	Windows, Linux, or macOS
Processor	Modern dual-core processor or equivalent
RAM	2 GB minimum recommended
Language	Python 3.x
GUI	Tkinter
Storage	Local JSON file
IDE	IDLE, VS Code, PyCharm, or any text editor

4. PYTHON CONCEPTS USED

4.1 Lists

The websites variable is a List containing multiple website records. New records are appended with append(), records are searched by iteration, and selected records are removed with pop().

4.2 Dictionaries

Each website is stored as a Dictionary with keys such as name, url, category, and notes. Key-value pairs make the data readable and easy to update.

4.3 File Handling

The program uses open() with UTF-8 encoding to read and write websites.json. The json module converts Python data to JSON and back. Exception handling covers missing, unreadable, or invalid files.

4.4 Tkinter

Tkinter creates the main window, labels, entry fields, buttons, and Treeview table. Button commands call functions to add, search, delete, and refresh records.

5. SYSTEM DESIGN AND ALGORITHM

Main Modules

- Input Module: accepts website details.
- Record Module: stores records as Dictionaries in a List.
- Storage Module: loads and saves websites.json.
- Search Module: filters records by text.
- Display Module: refreshes the Treeview table.
- Delete Module: removes a selected record and saves changes.

Workflow / Algorithm

1. Start the application and create the Tkinter window.
2. Load existing records from websites.json, if available.
3. Enter website details and click Add Website.
4. Validate required fields and create a Dictionary.
5. Append the record to the List and save the JSON file.
6. Search records by name, URL, category, or notes when needed.
7. Select a row and confirm deletion to remove a record.
8. Refresh the table and close the application; saved data remains on disk.

Example Data Structure

```
websites = [
    {
        "name": "Python Documentation",
        "url": "https://docs.python.org/3/",
        "category": "Education",
        "notes": "Official Python reference"
    }
]
```

6. IMPLEMENTATION / SOURCE CODE

Save the following code as website_data_tracker.py and run it with Python 3. The program creates websites.json in the same folder when data is saved.

```
import json
import os
import tkinter as tk
from tkinter import ttk, messagebox

DATA_FILE = "websites.json"
websites = []

def load_data():
    global websites
    if not os.path.exists(DATA_FILE):
        websites = []
        return
    try:
        with open(DATA_FILE, "r", encoding="utf-8") as file:
            data = json.load(file)
        websites = data if isinstance(data, list) else []
    except (OSError, json.JSONDecodeError):
        websites = []
        messagebox.showwarning("Data Warning",
            "Saved data could not be read. Starting with an empty list.")

def save_data():
    try:
        with open(DATA_FILE, "w", encoding="utf-8") as file:
            json.dump(websites, file, indent=4, ensure_ascii=False)
    except OSError as error:
        messagebox.showerror("Save Error", str(error))

def refresh_table(records=None):
    for item in table.get_children():
        table.delete(item)
    shown = records if records is not None else list(enumerate(websites))
    for index, record in shown:
        table.insert("", "end", iid=str(index), values=(
            record.get("name", ""), record.get("url", ""),
            record.get("category", ""), record.get("notes", "")))

def clear_fields():
    name_var.set("")
    url_var.set("")
    category_var.set("")
    notes_var.set("")
    name_entry.focus_set()

def add_website():
    name = name_var.get().strip()
    url = url_var.get().strip()
    category = category_var.get().strip() or "General"
    notes = notes_var.get().strip()
    if not name or not url:
        messagebox.showwarning("Missing Information",
                               "Enter both website name and URL.")
        return
    websites.append({"name": name, "url": url,
```

```
                    "category": category, "notes": notes})
    save_data()
    refresh_table()
    clear_fields()
    messagebox.showinfo("Added", "Website record added successfully.")

def search_websites(*args):
    query = search_var.get().strip().lower()
    matches = []
    for index, record in enumerate(websites):
        if not query or any(query in str(value).lower()
                            for value in record.values()):
            matches.append((index, record))
    refresh_table(matches)

def delete_website():
    selected = table.selection()
    if not selected:
        messagebox.showwarning("No Selection",
                               "Select a website record first.")
        return
    index = int(selected[0])
    if messagebox.askyesno("Confirm Delete",
                           "Delete the selected website?"):
        websites.pop(index)
        save_data()
        refresh_table()
        messagebox.showinfo("Deleted", "Website record deleted.")

root = tk.Tk()
root.title("Website Data Tracker")
root.geometry("980x620")
root.minsize(780, 500)

ttk.Label(root, text="WEBSITE DATA TRACKER",
          font=("Arial", 18, "bold")).pack(pady=12)

form = ttk.Frame(root, padding=10)
form.pack(fill="x")
name_var = tk.StringVar()
url_var = tk.StringVar()
category_var = tk.StringVar()
notes_var = tk.StringVar()
search_var = tk.StringVar()
search_var.trace_add("write", search_websites)

ttk.Label(form, text="Website Name").grid(row=0, column=0, sticky="w", padx=5, pady=5)
name_entry = ttk.Entry(form, textvariable=name_var, width=28)
name_entry.grid(row=0, column=1, sticky="ew", padx=5, pady=5)
ttk.Label(form, text="Website URL").grid(row=0, column=2, sticky="w", padx=5, pady=5)
ttk.Entry(form, textvariable=url_var, width=35).grid(row=0, column=3, sticky="ew", padx=5, pady=5)
ttk.Label(form, text="Category").grid(row=1, column=0, sticky="w", padx=5, pady=5)
ttk.Entry(form, textvariable=category_var).grid(row=1, column=1, sticky="ew", padx=5, pady=5)
ttk.Label(form, text="Notes").grid(row=1, column=2, sticky="w", padx=5, pady=5)
ttk.Entry(form, textvariable=notes_var).grid(row=1, column=3, sticky="ew", padx=5, pady=5)
form.columnconfigure(1, weight=1)
form.columnconfigure(3, weight=1)

buttons = ttk.Frame(root, padding=(10, 0))
buttons.pack(fill="x")
ttk.Button(buttons, text="Add Website", command=add_website).pack(side="left", padx=4)
```

```
ttk.Button(buttons, text="Delete Selected", command=delete_website).pack(side="left", padx=4)
ttk.Button(buttons, text="Clear Fields", command=clear_fields).pack(side="left", padx=4)

search_frame = ttk.Frame(root, padding=10)
search_frame.pack(fill="x")
ttk.Label(search_frame, text="Search records:").pack(side="left", padx=5)
ttk.Entry(search_frame, textvariable=search_var).pack(side="left", fill="x", expand=True)

table_frame = ttk.Frame(root, padding=10)
table_frame.pack(fill="both", expand=True)
columns = ("name", "url", "category", "notes")
table = ttk.Treeview(table_frame, columns=columns, show="headings", selectmode="browse")
for col, label, width in [
    ("name", "Website Name", 180), ("url", "URL", 280),
    ("category", "Category", 120), ("notes", "Notes", 250)]:
    table.heading(col, text=label)
    table.column(col, width=width)
scroll = ttk.Scrollbar(table_frame, orient="vertical", command=table.yview)
table.configure(yscrollcommand=scroll.set)
table.pack(side="left", fill="both", expand=True)
scroll.pack(side="right", fill="y")

load_data()
refresh_table()
root.mainloop()
```

7. TESTING AND SAMPLE OUTPUT

Test Case	Action	Expected Result
Add record	Enter name and URL; click Add	Record appears and is saved
Missing input	Leave name or URL blank	Warning message is displayed
Search	Type part of a name/category	Matching records are shown
Delete	Select a row and confirm	Selected record is removed
Persistence	Close and reopen the app	Saved records load again
Clear search	Remove search text	All records are displayed

Sample Record

Website Name	URL	Category	Notes
Python Documentation	https://docs.python.org/3/	Education	Official Python reference

Testing note: Run the program on the target computer and record the actual result for each test case before submission.

8. ADVANTAGES AND LIMITATIONS

Advantages

- Easy-to-use graphical interface.
- Uses built-in Python modules and a local JSON file.
- Demonstrates Lists, Dictionaries, File Handling, and Tkinter.
- Search and delete functions make records easier to manage.

Limitations

- Data is stored only on the local computer.
- The program does not verify website availability.
- The JSON file is plain text and is not encrypted.
- No multi-user collaboration or cloud synchronization.

9. FUTURE ENHANCEMENTS

- Add edit/update functionality and duplicate URL detection.
- Validate URLs and optionally check website availability.
- Add sorting, favorites, tags, and date-added fields.
- Export records to CSV or Excel.
- Use SQLite for larger datasets and add optional encrypted storage.

10. CONCLUSION AND REFERENCES

Website Data Tracker demonstrates how core Python concepts can be combined to build a practical desktop application. Lists manage the collection, Dictionaries represent each website, File Handling saves information to JSON, and Tkinter provides the graphical interface. The project offers a useful foundation for future improvements such as editing, URL validation, and database storage.

- Python 3 Documentation: https://docs.python.org/3/
- Tkinter Documentation: https://docs.python.org/3/library/tkinter.html
- JSON Module: https://docs.python.org/3/library/json.html
- Python Data Structures: https://docs.python.org/3/tutorial/datastructures.html